



TP1 : Méthodes Globales d'Interprétabilité : Partial Dependence Plots (PDP)-Partie 1

Objectifs

Charger le dataset présenté en cours et créer des Partial Dependence Plots (PDP) pour différents modèles.

Partie 0 : Installation et import des packages nécessaires

```
## Packages nécessaires
import sys
!pip install lux-explainer
!pip install interpret
!pip install git+https://github.com/christophM/rulefit.git
!pip install git+https://github.com/sbobek/ALEPython.git@dev#egg=alepython
!pip install eli5

import pandas as pd
import re
import graphviz
from sklearn.datasets import load_iris
from graphviz import Source
from IPython.display import SVG, Image
from interpret import set_visualize_provider
from interpret.provider import InlineProvider

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score, mean_squared_error
import numpy as np
from interpret.glassbox import ExplainableBoostingRegressor
from interpret import show
from matplotlib import pyplot as plt
from alepython import ale_plot
import eli5

set_visualize_provider(InlineProvider())
```

Ce que vous devez faire :

Un exemple de PDP avec une **régression linéaire** est fourni ci-dessous (**pas très utile mais sert de démonstration**).



Question 1 : Interprétez les PDP obtenus.

- Que dit l'effet de la température ?
- L'humidité augmente-t-elle ou diminue-t-elle la demande de vélos ?
- Le graphique 2D montre-t-il une interaction ?

Question 2 : Produisez les PDP pour les modèles suivants :

Vous devez reproduire le même code mais en remplaçant le modèle par :

1. **XGBoost Regressor**
2. **DecisionTreeRegressor**
3. **SVR(kernel="rbf")**
4. **MLPRegressor**

Comparez les formes des PDP :

- Quelle méthode produit les effets les plus non linéaires ?
- Quel modèle semble le plus sensible aux interactions ?
- Quelle visualisation est la plus facile à interpréter ?

1. Chargement du dataset

```
from sklearn.datasets import fetch_openml

bikes = fetch_openml("Bike_Sharing_Demand", version=2, as_frame=True, parser="pandas")
# Copie explicite pour éviter le warning "SettingWithCopyWarning"
X, y = bikes.data.copy(), bikes.target

# On utilise seulement un sous-ensemble du dataset pour accélérer l'exemple
X = X.iloc[:5, :]
y = y[:5]
```

2. Prétraitement du dataset

```
# Il n'y a qu'un seul enregistrement "heavy_rain", on le remplace par "rain"
X["weather"].replace(to_replace="heavy_rain", value="rain", inplace=True)

# Les données couvrent 2 ans : la première année = entraînement, la seconde = test
mask_training = X["year"] == 0.0
X = X.drop(columns=["year"])
X_train, y_train = X[mask_training], y[mask_training]
X_test, y_test = X[~mask_training], y[~mask_training]
```



```
numerical_features = [
    "temp",
    "feels_like",
    "humidity",
    "windspeed",
]

categorical_features = X_train.columns.drop(numerical_features)
```

```
from sklearn.linear_model import LinearRegression
from time import time
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OrdinalEncoder
from sklearn.pipeline import make_pipeline
from sklearn.inspection import PartialDependenceDisplay

print("Calcul des graphiques de dépendance partielle...")

features_info = {
    "features": ["temp", "humidity", ("temp", "humidity")],
    "kind": "average",
}
```



```
common_params = {
    "subsample": 50,
    "n_jobs": 2,
    "grid_resolution": 20,
    "random_state": 0,
}

_, ax = plt.subplots(ncols=3, figsize=(10, 4), constrained_layout=True)
tic = time()

# ColumnTransformer pour transformer les variables
lr_preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OrdinalEncoder(), categorical_features),
        ("num", "passthrough", numerical_features),
    ],
    sparse_threshold=1,
    verbose_feature_names_out=False,
).set_output(transform="pandas")

lrppl = make_pipeline(
    lr_preprocessor,
    LinearRegression(),
)

lrppl.fit(X_train, y_train)

display = PartialDependenceDisplay.from_estimator(
    lrppl,
    X_train,
    **features_info,
    ax=ax,
    **common_params,
)

print(f"terminé en {time() - tic:.3f}s")

_ = display.figure_.suptitle(
    "PDP unidimensionnels et bidimensionnels (régression linéaire)", fontsize=16
)
```



1-way vs 2-way of numerical PDP using linear regression

